Tema 3: Subsistema de Entrada/Salida

Conceptos Fundamentales, Métodos de Control y Análisis Práctico

GUÍA DE ESTUDIO Y RESUMEN EJECUTIVO • ARQUITECTURA DE COMPUTADORES

El subsistema de **Entrada/Salida (E/S)** es la interfaz que permite al computador interactuar con el mundo exterior (periféricos como discos, teclados, sensores y pantallas). Su diseño busca resolver dos problemas clave: la disparidad de velocidades entre la CPU y los periféricos, y la diferencia en los formatos de datos.

1. Espacios de Direccionamiento de E/S

Para comunicarse con los periféricos, la CPU debe acceder a sus registros internos (generalmente divididos en registros de *Datos*, *Estado* y *Control*). Existen dos metodologías principales para organizar estas direcciones:

- **E/S Mapeada en Memoria (Memory-Mapped I/O):** Los registros del periférico comparten el mismo espacio de direccionamiento que la memoria principal. La CPU utiliza las mismas instrucciones de acceso a memoria (por ejemplo, `lbu`, `sb`, `lw`, `sw` en RISC-V).
  Ventaja: No requiere instrucciones específicas, simplificando el repertorio (ISA).
  Inconveniente: Consume parte del mapa de memoria direccionable.
- **E/S Separada o Aislada (Isolated I/O):** Los periféricos disponen de un espacio de direcciones completamente propio e independiente de la memoria principal.
  Ventaja: Preserva todo el mapa de memoria para programas y datos.
  Inconveniente: Requiere instrucciones exclusivas de hardware para comunicarse con los puertos (como `inb` y `outb` en arquitecturas x86).

2. Métodos de Control de la Transferencia de Datos

Existen tres mecanismos fundamentales para gestionar el intercambio de datos entre el procesador y los periféricos, ordenados de menor a mayor eficiencia en el uso de la CPU:

A. E/S Programada por Consulta (Polling o Espera Activa)

Es el método más sencillo. La CPU asume el control total de la operación. Ejecuta un bucle cerrado en el que lee continuamente el **registro de estado** del periférico hasta que un bit determinado (como *Ready*, *Disponible* o *Listo*) cambia de valor, indicando que la transferencia puede realizarse.

> **Análisis de Eficiencia**
>
> Provoca un alto desperdicio de ciclos de reloj. Si el dispositivo es lento (por ejemplo, una impresora o un sensor térmico), la CPU pasa la mayor parte del tiempo atrapada en el bucle de espera, reduciendo drásticamente el rendimiento global del sistema.

B. E/S por Interrupciones

Elimina la necesidad de la espera activa. La CPU inicia la operación o simplemente continúa con su flujo de trabajo normal (ejecutando otras tareas). Cuando el periférico está listo para enviar o recibir datos, activa una línea física de **interrupción** conectada al procesador.

Al recibir la señal, el procesador finaliza la instrucción en curso y realiza los siguientes pasos:

1. **Cambio de Contexto:** Salva los registros esenciales y el Contador de Programa (*PC*) actual en la pila para no perder el estado del hilo de ejecución principal.
2. **Rutina de Servicio de Interrupción (RSI / RTI):** Salta a una dirección de memoria específica (obtenida mediante un vector de interrupción) donde se ejecuta el código que atiende al periférico (lectura o escritura del dato).
3. **Retorno:** Restaura el contexto guardado y reanuda la ejecución del programa exactamente donde se interrumpió.

C. Acceso Directo a Memoria (DMA - Direct Memory Access)

Para periféricos de alta velocidad (como discos de almacenamiento o escáneres masivos), las interrupciones individuales por cada byte saturarían a la CPU debido al coste del cambio de contexto. El método **DMA** delega el control de la transferencia a un chip especializado: el **Controlador de DMA (DMAC)**.

El proceso sigue este esquema secuencial:

1. Configuración: La CPU indica al DMAC la dirección base de memoria, el volumen de datos (número de bytes) y la dirección del movimiento (lectura/escritura).
2. Transferencia autónoma: El DMAC gestiona de forma directa los buses de datos, direcciones y control, moviendo bloques completos entre el periférico y la memoria principal (mediante técnicas como "robo de ciclos" de bus).
3. Finalización: Al completarse todo el bloque, el DMAC genera *una única interrupción* para avisar a la CPU de que los datos ya están consolidados.

3. Operaciones de Bitwise (Manipulación de Bits) en E/S

Dado que los registros de control y estado agrupan múltiples señales lógicas en una sola palabra, es imprescindible dominar las operaciones de bits para interactuar con ellos sin corromper el resto de funciones:

Operación	Propósito en E/S	Ejemplo en C / Explicación	Ejemplo en RISC-V
AND (`&`)	Aislar/Consultar un bit de estado.	`estado & 0x04` (Comprueba si el bit 2 está activo).	`andi x13, x12, 0x40`
OR (`|`)	Activar uno o varios bits de control.	`control | 0x01` (Pone a 1 el bit 0 sin alterar los demás).	`ori x15, x15, 0x01`
AND-NOT (`& ~`)	Desactivar un bit específico.	`control & ~0x10` (Fuerza a 0 el bit 4).	Usa máscara inversa (ej: `0xDF` para apagar bit 5).
XOR (`^`)	Conmutar/Invertir el valor de un bit.	`reg ^ 0x04` (Si el bit 2 era 0 pasa a 1, y viceversa).	`xori x5, x5, 0x04`

4. Guía Práctica de Código (Patrones Comunes del Boletín)

Patrón 1: Espera Activa (Polling) en Ensamblador RISC-V (E/S Mapeada)

El siguiente fragmento ilustra cómo consultar el bit 6 de un registro de estado situado en la dirección base contenida en `x10` hasta que se ponga a cero (lógica inversa o dispositivo ocupado):

```
# x10 contiene la dirección base del dispositivo (ej: 0x100)
bucle_espera:
    lbu  x12, 0(x10)        # Lee el Registro de Estado (desplazamiento 0)
    andi x13, x12, 0x40     # Aplica máscara para aislar el Bit 6 (0x40 = 01000000b)
    bne  x13, x0, bucle_espera # Si el resultado NO es cero (bit activo), sigue
esperando

    lbu  x20, 2(x10)        # Una vez libre, lee el Registro de Datos (desplazamiento
2)
```

Patrón 2: Configuración y Lectura por Polling en Lenguaje C (E/S Separada)

Un patrón recurrente en el boletín consiste en activar señales de lectura y esperar la confirmación de hardware mediante funciones de bajo nivel (`inb` / `outb`):

```
// Función para leer un sensor específico [0, 1, 2] de un dispositivo
unsigned char leer_sensor_polling(unsigned char num_sensor) {
    // 1. Configurar el sensor a leer en los bits correspondientes y activar la señal
LEER (bit activo en baja, p.ej. 0)
    // Supongamos un puerto de control en 0x31
    unsigned char config = (num_sensor << 4) & ~0x08; // Desplazar ID y poner LEER* a
0
    outb(0x31, config);

    // 2. Espera activa hasta que el bit LEÍDO* (bit 2, activo en baja) se ponga a 0
    while ((inb(0x32) & 0x04) != 0);

    // 3. Devolver el dato obtenido desde el puerto de datos (0x30)
    return inb(0x30);
}
```

5. Modelado del Rendimiento: E/S Programada vs Interrupciones vs DMA

Uno de los pilares analíticos del tema consiste en calcular matemáticamente el impacto de cada técnica en los ciclos de la CPU. Para tomar la decisión de diseño óptima, se consideran las siguientes variables:

- **Frecuencia del Periférico (f_p):** Cantidad de bytes o transferencias requeridas por segundo.
- **Coste de Transferencia Directa (C_t):** Instrucciones o ciclos dedicados estrictamente a mover el dato.
- **Coste de Cambio de Contexto (C_{cc}):** Ciclos invertidos por la CPU en salvar y restaurar registros durante una interrupción.
- **Coste de Inicialización de DMA (C_{init}):** Ciclos empleados en configurar los registros del DMAC.

Mecanismo	Ecuación de Coste CPU por Bloque	Criterio de Aplicación
Polling	$Ciclos = N \times (Espera + C_t)$	Dispositivos ultrarrápidos o sistemas dedicados de tiempo real simple.
Interrupciones	$Ciclos = N \times (C_{cc} + C_t)$	Transferencias esporádicas, teclados o flujos lentos de datos.
DMA	$Ciclos = C_{init} + C_{cc_final}$	Transferencia masiva de bloques (Discos duros, tarjetas de red, escáneres).

Regla de Oro del Diseñador de Arquitecturas

A medida que aumenta el tamaño del bloque (N) o la tasa de transferencia, el coste del **DMA** se mantiene prácticamente constante, logrando una aceleración exponencial respecto a las interrupciones, cuyo coste escala linealmente con cada elemento transferido.